

HW2

侯昱晨 521030910340

11 27 2023

1 Writing Assignment

$$1. a. \textcircled{1} \frac{\partial E}{\partial T} = -2 \sum_{i=1}^N (Y_i - AX_i - T) = 0$$

$$\therefore T = \frac{1}{N} \sum_{i=1}^N (Y_i - AX_i)$$

$$= \bar{Y} - A^T \bar{X}$$

$$\textcircled{2} \frac{\partial E}{\partial A} = E(A) = \sum_{i=1}^N \|Y_i - AX_i - T^*\|^2$$

$$= \sum_{i=1}^N \| (Y_i - \bar{Y}) - A(X_i - \bar{X}) \|^2$$

$$= \sum_{i=1}^N \| Y_i' - AX_i' \|^2$$

$$\frac{\partial E(A)}{\partial A} = 2X(X^TAX - Y^T) = 0$$

$$\therefore A^* = (Y'X'^T)(XX'^T)^{-1}$$

$$X' = [X_1 - \bar{X}, \dots, X_N - \bar{X}]$$

$$Y' = [Y_1 - \bar{Y}, \dots, Y_N - \bar{Y}]$$

b. λA 与 A 效果相同 $\rightarrow A$ 有 8 个自由度

4 个点 $\rightarrow$ 8 个坐标方程

T^* 由 A^* 计算

故 4 个点可计算 $[A, T]$

2 Programming Assignment

2.1 find_image_points

init_world_points 为 (pattern_size) 分配棋盘格 world_points。
调用 cv2.findChessboardCorners 获得像素坐标 image_points

2.2 calibrate_camera

在每张图片中，使用 centered(world_points, image_points) 筛选出位于 2D 图像中心区域的点对。

参数 center_fraction 调节中心区域的大小，本实验中笔者设置为 0.8，即考虑图片靠中心 80% 的棋盘格点。

对每张图片分别使用中心点对构建线性方程，使用 SVD 分解/特征向量求出单应性矩阵 H 并放缩至 H[2,2] 为 1。(见 **Figure: 1**) 两种方法对本实验的影响不大。

```
for j in range(13):
    ct_world_points, ct_image_points = centered(world_points[j], image_points[j])
    M = np.zeros((2*ct_world_points.shape[0], 9))
    for i in range(ct_world_points.shape[0]):
        X, Y, Z = ct_world_points[i]
        u, v = ct_image_points[i]
        M[i * 2] = [X, Y, 1, 0, 0, 0, -u * X, -u * Y, -u]
        M[i * 2 + 1] = [0, 0, 0, X, Y, 1, -v * X, -v * Y, -v]

    eigenvalues, eigenvectors = np.linalg.eigh(np.dot(M.T, M))
    min_index = np.argmin(eigenvalues)
    H = eigenvectors[:, min_index].reshape((3,3))

    # U,S,Vt=np.linalg.svd(M)
    # H = Vt[-1].reshape(3, 3)

    H = H/H[2,2]
```

Figure 1: 投影矩阵

整合所有图片的单应性矩阵，使用张正友标定法得到相机内参矩阵 K。(见 **Figure: 2a**)

利用投影矩阵与内参矩阵为每张图片分别求解外参矩阵，由于棋盘格的 3D 坐标中 z 维度为 0，第三个旋转向量由前两个求正交积得到。(见 **Figure: 2b**)

2.3 reprojection_error

利用内参矩阵与外参矩阵重新投影 world_points，计算 reprojected_points 与 image_points 的欧氏距离平均值，作为偏移误差。

对比笔者的标定方法与 opencv 的标定方法的偏移误差结果如 **Figure: 3**所示。

```

A.extend(
    [
        [H[0, 0] * H[0, 0] - H[0, 1] * H[0, 1],
         2 * (H[0, 0] * H[1, 0] - H[0, 1] * H[1, 1]),
         2 * (H[0, 0] * H[2, 0] - H[0, 1] * H[2, 1]),
         H[1, 0] * H[1, 0] - H[1, 1] * H[1, 1],
         2 * (H[1, 0] * H[2, 0] - H[1, 1] * H[2, 1]),
         H[2, 0] * H[2, 0] - H[2, 1] * H[2, 1]],

        [
            H[0,0] * H[0, 1],
            H[0, 0] * H[1, 1] + H[0, 1] * H[1, 0],
            H[0, 0] * H[2, 1] + H[0, 1] * H[2, 0],
            H[1, 0] * H[1, 1],
            H[1, 0] * H[2, 1] + H[1, 1] * H[2, 0],
            H[2, 0] * H[2, 1]
        ]
    ]
)

A = np.array(A)
eigval, eigvec = np.linalg.eig(A.T@A)
b = eigvec[:, np.argmin(eigval)]
b /= np.linalg.norm(b)

B = np.array([
    [b[0], b[1], b[2]],
    [b[1], b[3], b[4]],
    [b[2], b[4], b[5]],
])

K = np.linalg.cholesky(B)
K = np.linalg.inv(K.T)
K /= K[2,2]

external_matrixs = []
for i in range(13):
    E = np.zeros((3, 4))
    R_temp = np.linalg.inv(K)@P[i]
    E[:3, 0] = R_temp[:, 0]
    E[:3, 1] = R_temp[:, 1]
    E[:3, 3] = R_temp[:, 2]
    E[:3, 2] = np.cross(E[:3, 0], E[:3, 1])
    external_matrixs.append(E)

```

(a) 内参矩阵

(b) 外参矩阵

Figure 2: 内外参数分解

Camera Calibration Matrix:

```

[[535.94 -2.75 353.91]
 [ 0.    537.86 236.79]
 [ 0.     0.     1.   ]]
Image 0 offset_errors: 0.7853403859527488
Image 1 offset_errors: 1.1751878987116169
Image 2 offset_errors: 1.7455327403642686
Image 3 offset_errors: 1.2901879580023659
Image 4 offset_errors: 1.452527534738405
Image 5 offset_errors: 1.2570491902325645
Image 6 offset_errors: 0.7689067327587515
Image 7 offset_errors: 1.2673964568035618
Image 8 offset_errors: 0.8360607210227696
Image 9 offset_errors: 1.0910161276112678
Image 10 offset_errors: 1.3572520543371354
Image 11 offset_errors: 0.6326517276578753
Image 12 offset_errors: 1.0744716196056219

```

(a) 作者的相机标记偏移误差

Camera Calibration Matrix by OpenCV:

```

Camera Matrix:
[[531.15  0.    341.82]
 [ 0.    531.43 235.04]
 [ 0.     0.     1.   ]]
Image 0 offset_errors: 2.6197422
Image 1 offset_errors: 2.6043782
Image 2 offset_errors: 4.9476876
Image 3 offset_errors: 2.9030893
Image 4 offset_errors: 3.7653995
Image 5 offset_errors: 7.709775
Image 6 offset_errors: 2.731358
Image 7 offset_errors: 2.6537378
Image 8 offset_errors: 2.0513926
Image 9 offset_errors: 2.4226408
Image 10 offset_errors: 3.0254703
Image 11 offset_errors: 1.5936007
Image 12 offset_errors: 2.4996717

```

(b) OpenCV 的相机标记偏移误差

Figure 3: 相机标记偏移误差比较